

FITBUDDY – AI FITNESS PLAN GENERATOR USING GEMINI MODELS

3. Project Design Phase

The Project Design Phase converts the identified requirements into a technical design for the FitBuddy application. The main objective of this phase is to decide how different components of the system will communicate with each other.

FitBuddy follows a web-based architecture consisting of a frontend, backend, database, and AI layer. The frontend provides the user interface, while the FastAPI backend handles application logic and communication between the frontend, database, and Gemini AI models.

The frontend is implemented using HTML, CSS, and Jinja2 templates. The main pages include the home/input page, workout result page, and user/admin-related views. The user enters fitness information through the web form.

The backend is implemented using FastAPI. It provides routes for generating workout plans, submitting feedback, accessing users, and managing application requests. FastAPI receives the user's input, validates it, and sends the required information to the appropriate AI module.

The AI layer is divided into separate modules. One Gemini-based module is responsible for generating workout plans. Another module generates nutrition and recovery recommendations. A separate module processes user feedback and generates an updated workout plan.

SQLite is used as the database, with SQLAlchemy providing database interaction. The database stores user information, original workout plans, updated workout plans, and nutrition tips.

The basic architecture can be represented as:

User → Frontend → FastAPI Backend → Gemini AI → Database → Result

The main components are:

- Frontend Layer – collects input and displays results.
- FastAPI Layer – handles application requests and business logic.
- AI Layer – generates workout, nutrition, and updated plans.
- Database Layer – stores user and plan information.
- Configuration Layer – manages API keys and application settings.

The design also includes a demo mode so that the application can be tested without making an actual Gemini API request. This helps during development and testing.

The modular design makes the project easier to maintain because workout generation, nutrition generation, feedback processing, database operations, and web routes are separated into different components.